

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 11 · LECTURE NOTES

Randomness in Distributed Systems

Where a coin is not an optimisation but a necessity.

MODULE OUTCOMES

- Explain symmetry breaking and why determinism fails.
- Explain randomised consensus and the FLP circumvention.
- Explain gossip protocols and their convergence.
- Explain randomised load balancing in a distributed setting.
- Explain randomness in failure detection and backoff.

1 Symmetry breaking

IDENTICAL PROCESSES CANNOT BREAK SYMMETRY DETERMINISTICALLY

Two identical processes in identical states, receiving identical messages, will take identical actions. So neither can be selected as leader by any deterministic algorithm, because the two executions are identical step for step.

And this is a genuine impossibility rather than a difficulty — there is no cleverer deterministic algorithm, because the executions are indistinguishable by construction and any algorithm is a function of the execution.

A single coin flip breaks it immediately. Each process flips; with probability $1 - 2^{-k}$ after k rounds the two have differed at least once — so the problem is solved with high probability in a constant expected number of rounds, which is as good as it could be.

Which is the cleanest case in the entire subject of randomness being *necessary* rather than merely convenient — Module 01 §1's first use (defeating an adversary, or here defeating a symmetry) in its purest form, with a provable impossibility on the other side and no deterministic alternative to compare against.

AND THE SAME ARGUMENT GIVES RANDOMISED BACKOFF

Two senders that collide on a shared medium will collide again if they both retry deterministically — the retry preserves the symmetry that caused the collision, which is the same argument as the callout above.

So retry after a random delay. Exponential backoff doubles the range of the random delay after each collision, which adapts the amount of randomness to the observed level of contention rather than fixing it in advance.

And it is in Ethernet, in TCP's congestion response, in every HTTP client's retry logic, and in every distributed lock implementation — which makes randomised exponential backoff the most widely deployed randomised algorithm in existence, by a very large margin.

With one practical refinement that is frequently omitted: add jitter. Backoff with a deterministic schedule re-synchronises clients into waves — all the clients that failed together retry together, fail together, and retry together again — **which is the thundering-herd failure, and is why the standard published guidance is 'exponential backoff with jitter'** rather than exponential backoff alone. **The jitter is doing the symmetry breaking and the exponential part is doing the congestion control, and they are two separate mechanisms.**

2 Consensus

FLP SAYS DETERMINISTIC CONSENSUS IS IMPOSSIBLE; RANDOMISATION ESCAPES IT

Fischer, Lynch and Paterson: in an asynchronous system with even a single crash failure, no deterministic algorithm achieves consensus (CSCE 678 Module 03's central result).

The proof constructs an infinite execution that never decides — not a wrong answer, but an indefinite delay, by repeatedly extending a bivalent configuration. **The impossibility is about liveness rather than safety.**

And a randomised algorithm escapes it: Ben-Or's protocol terminates with probability 1, in an expected constant number of rounds (for a small enough failure fraction) — because the adversarial execution that FLP constructs has probability zero under the protocol's own randomisation. The adversary cannot steer the coin flips.

Which is the precise form of the escape, and it is worth stating exactly: randomisation does not contradict FLP — it changes the guarantee from 'always terminates' to 'terminates with probability 1', and the impossibility result was about the first. So the theorem and the protocol are both correct, which is a useful thing to be able to explain when someone points out the apparent contradiction.

	Deterministic	Randomised
Safety (never decide inconsistently)	Guaranteed.	Guaranteed — unchanged. See the note.

	Deterministic	Randomised
Liveness (eventually decide)	Impossible in the asynchronous model with one failure (FLP).	With probability 1.
Round complexity	Unbounded in the worst case — that is the content of FLP.	Expected $O(1)$ for small failure fractions; still unbounded in the worst case.
What is used in practice	Paxos and Raft — which are deterministic and use <i>timeouts</i> to sidestep FLP by assuming partial synchrony (and randomised election timeouts, which is this module again).	Ben-Or, HoneyBadger, and several blockchain consensus protocols, where asynchrony cannot be assumed away.

Note that safety is never traded. A randomised consensus protocol never decides inconsistently — the randomness affects only *whether and when* it decides, **which is the right place for it** and is the point practitioners most need to hear. **And note that Raft's randomised election timeouts are this module's symmetry breaking inside an ostensibly deterministic protocol**, which is worth spotting.

3 Gossip and load balancing

GOSSIP SPREADS INFORMATION IN $O(\log n)$ ROUNDS WITH CONSTANT WORK PER NODE

Each node, in each round, picks a random peer and exchanges state with it. Information reaches every node in $O(\log n)$ rounds with high probability — the same doubling argument as an epidemic.

Which is Module 06's random graph connectivity result in a different guise — the union of the random peer choices over several rounds forms a well-connected expanding graph, and the $O(\log n)$ is its diameter.

And the work per node per round is constant, which is what makes it scalable: **no node needs a global membership view, a coordinator, or a consistent snapshot of anything.**

So it is used for membership, failure detection, configuration propagation, and anti-entropy repair — Cassandra, Consul, and Serf are all built on gossip (CSCE 678 Module 04). **And the robustness to failure is the selling point rather than the speed:** a gossip protocol degrades gracefully as nodes fail, because there is nothing central to lose.

- **Module 03 §2's power of two choices, applied to servers: query two backends at random and send the request to whichever reports less load.**
- **Which gives $\log \log n$ maximum load with no coordination whatsoever** — no central balancer, no global state, two probes per request.
- **And the practical caveat is stale information: if every load balancer queries the same two backends and they all see the same stale load figures, they all pick the same one, and the advantage inverts into a disadvantage** — the chosen backend receives everything. **This is the herd-behaviour failure, and it has been observed in production repeatedly.**
- **So the load must be sampled freshly, or the two choices must be randomised independently per balancer, or an age-weighted estimate used** — **and the theorem's assumption that each ball sees the current load is the one that a distributed system usually violates.**
- **And consistent hashing with bounded loads** combines the two approaches: a hash determines each request's preferred backend, and overflow spills to the next in the ring — **which gives locality (useful for caching) and bounded load together.**

4 The remaining uses

- **Failure detection. Randomised probe targets avoid correlated false positives** — if every node probes the same monitor, a single slow monitor marks everything dead — **and SWIM's random-peer probing with indirect confirmation is the standard design.**
- **Sampling for monitoring and tracing. Distributed trace sampling is reservoir sampling** (Module 05 §3) **applied to requests**, and head-based against tail-based sampling is a question about which distribution you want.
- **Randomised leader rotation**, which spreads the leadership load and **limits the value of attacking or predicting any single leader** — a security property as much as a performance one.

- **Chaos engineering.** Randomly terminating instances to test resilience is **sampling the failure space rather than attempting to enumerate it**, which is the only tractable approach given the number of possible failure combinations.
- **And cryptographic nonces, session identifiers, and tokens, where the randomness is a security requirement rather than a performance optimisation (CSCE 711) — and this is the one case where a weak source is catastrophic rather than merely degrading (Module 13 §2).**

WHY THIS MODULE MATTERS

This is the setting where randomness is most clearly *necessary* rather than merely convenient — symmetry breaking (§1) and asynchronous consensus (§2) are both **impossibility results that randomisation escapes**, and there is no deterministic alternative to weigh it against.

And the escapes are precise rather than hand-waved: the guarantee changes from 'always' to 'with probability 1', and the impossibility proof was about 'always'.

Which is a better justification for randomness than any efficiency argument, because **it is not a trade at all** — Module 01 §1's three uses each had a deterministic alternative to compare against, and here there is none.

And it means every distributed system you will ever build contains several of these — randomised backoff with jitter at an absolute minimum, and very probably gossip, two-choice balancing, randomised probing, and sampled tracing as well. Which makes this the module whose content you are most certain to have already deployed.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Fischer, Lynch & Paterson — Impossibility of Distributed Consensus with One Faulty Process (free)	§2's impossibility result — short, and the bivalent-configuration argument is worth following.
Ben-Or — Another advantage of free choice (free)	§2's randomised escape , in three pages.
Demers et al. — Epidemic Algorithms for Replicated Database Maintenance (free)	§3's gossip protocols , in the paper that introduced them to systems.
Brooker — Exponential Backoff and Jitter (free, AWS Architecture Blog)	§1's jitter requirement, measured — with the thundering-herd effect demonstrated and the variants compared.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Simulate two identical processes** attempting deterministic leader election, and confirm it never terminates.
- 2 **Add coin flips** and measure the expected number of rounds.
- 3 **Implement exponential backoff without jitter** for many clients and observe the retry waves.
- 4 **Add jitter** and measure the improvement in collision rate.
- 5 **Read the FLP proof** and identify the step a randomised protocol invalidates.
- 6 **Implement Ben-Or's protocol** and measure the rounds to decide against the failure fraction.
- 7 **Implement gossip** on 1000 nodes and measure the rounds to full propagation. Compare against $\log n$.
- 8 **Kill 30% of the nodes mid-propagation** and report what happens.
- 9 **Implement two-choice load balancing with stale load data** and reproduce the herd inversion.
- 10 **Fix it** and report which fix you used and why.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why can identical processes not break symmetry deterministically?
- 2 How does a coin flip escape it, and how fast?
- 3 Why does deterministic backoff fail, and what are jitter and exponential backoff each doing?
- 4 State FLP and explain precisely how randomisation escapes it.
- 5 What is never traded in randomised consensus?
- 6 Why does gossip take $O(\log n)$ rounds, and what other module does that come from?
- 7 What makes gossip scalable, and what is its real selling point?
- 8 Describe the two-choice herd inversion and its fixes.
- 9 Name five other uses of randomness in a distributed system.
- 10 Why is this the module whose content you have most certainly already deployed?